

Escuela Politécnica Nacional

Programación I

MANUAL DE TECNICO DEL PROYECTO

SEMESTRAL

FIS

Integrantes:

Derlis Ledesma

Nancy Insuasti

Docente:

Nelson Herrera

Período:

2025-B

ÍNDICE

1. Introducción.....	3
2. Arquitectura del software.....	3
3. Diseño de datos.....	4
4. Algoritmos críticos y lógica de negocio.....	5
5. Interfaz y experiencia de usuario.....	6
6. Seguridad y control de errores.....	6
7. Requisitos de despliegue.....	6

1. INTRODUCCIÓN

1.1. Propósito del Sistema

El SGCM es una solución de software diseñada para automatizar el flujo operativo de un centro de salud de nivel primario. Su objetivo es sustituir el agendamiento manual por un sistema digital que garantice la integridad de los datos, la unicidad de las historias clínicas y la optimización de los tiempos de los médicos.

1.2. Alcance

El sistema cubre los siguientes procesos de negocio:

Seguridad: Autenticación basada en roles Administrador, Médico, Paciente.

Gestión de Usuarios: Crear, Leer, Actualizar, Borrar a Pacientes y Médicos.

Operación Clínica: Agendamiento de citas validando disponibilidad y control de estados.

Inteligencia de Negocios: Generación de reportes planos para auditoría.

2. ARQUITECTURA DEL SOFTWARE

El sistema sigue un patrón de diseño Modular Monolítico. El código no está escrito en un solo archivo gigante, sino que se divide en responsabilidades claras.

2.1. Mapa de Archivos y Responsabilidades

Módulo	Responsabilidad Técnica	Dependencias
main.c	Orquestador principal. Maneja el ciclo de vida de la aplicación (<code>while(op!='q')</code>) y el enrutamiento de menús.	Todos los .h
auth.c	Motor de seguridad. Valida credenciales contra memoria temporal o estructuras cargadas.	auth.h

Módulo	Responsabilidad Técnica	Dependencias
pacientes.c	Gestión de memoria y persistencia del struct Paciente. Carga/Guarda en data_pacientes.txt.	pacientes.h, validaciones.h
medicos.c	Gestión del personal. Genera códigos automáticos (MED-XXX).	medicos.h
citas.c	Lógica transaccional. Vincula IDs de pacientes con códigos de médicos.	citas.h, medicos.h
validaciones.c	Librería auxiliar pura. Contiene algoritmos matemáticos y de limpieza de strings.	validaciones.h, <ctype.h>

3. DISEÑO DE DATOS

Aunque el sistema utiliza archivos planos, la lógica interna respeta un modelo relacional.

3.1. Entidades y Atributos

PACIENTE: almacenado en data_pacientes.txt

Primary Key: cedula (char[11]). Identificador único nacional.

nombre: Cadena de texto.

contrasena: Texto plano para autenticación. **Relación:**

Un Paciente puede tener N Citas

MÉDICO: Almacenado en data_medicos.txt

Primary Key: codigo (char[10]). Ejemplo: "MED-001".

especialidad: Define el área de atención. **Relación:** Un

Médico atiende N Citas

CITA: Almacenado en data_citas.txt

Primary Key: id (int). Autoincremental.

Foreign Key: id_paciente = Referencia a Paciente.cedula.

Foreign Key: codigo_medico = Referencia a Medico.codigo. **estado:**

Entero (0=Activa, 1=Cancelada). Permite borrado lógico.

4. ALGORITMOS CRÍTICOS Y LÓGICA DE NEGOCIO

Esta sección detalla *cómo* el sistema resuelve problemas complejos.

4.1. Algoritmo de Validación de Cédula

Ubicación: src/validaciones.c

Para garantizar datos reales, no se acepta cualquier número. El sistema implementa la validación oficial ecuatoriana:

Verificación de longitud y tipo: Debe tener 10 dígitos numéricos.

Validación de Provincia: Los dos primeros dígitos deben estar entre 01 y 24.

Algoritmo de Coeficientes:

Se multiplican los dígitos impares por 2 y los pares por 1.

Si el resultado es ≥ 10 , se resta 9.

Se suman los resultados.

El dígito verificador debe coincidir con la decena superior menos la suma.

4.2. Persistencia: Carga y Guardado

El sistema utiliza arreglos globales (db_pacientes[], db_citas[]) como memoria caché.

Al Iniciar (cargar_...): El sistema abre los archivos en modo lectura ("r"), parsea línea por línea usando strtok con el delimitador | y llena los arreglos.

Durante la Ejecución: Todas las búsquedas y validaciones se hacen en RAM (rápido).

Al Modificar (guardar_...): Inmediatamente después de cualquier registrar_ o cancelar_, el sistema vuelca todo el arreglo al disco, sobrescribiendo el archivo anterior. Esto minimiza el riesgo de pérdida de datos.

4.3. Generación Automática de IDs

Ubicación: src/medicos.c = registrar_medico()

Para evitar colisiones de claves primarias:

`int siguiente_num = total_medicos + 1; sprintf(nuevo.codigo, "MED-%03d", siguiente_num);` Este algoritmo garantiza que los códigos sigan el formato estándar MED-001, MED-002, facilitando la lectura y ordenamiento.

5. INTERFAZ Y EXPERIENCIA DE USUARIO

El sistema implementa una interfaz de línea de comandos robusta.

Limpieza de Pantalla: Uso estratégico de `system("cls")` para mantener el área de trabajo limpia entre transiciones de menú.

Captura de Datos Segura: Se utiliza una función propia `leer_texto` que encapsula `fgets` y elimina el salto de línea residual (`\n`), evitando los errores comunes de buffer sucio en C.

Navegación: Menús cíclicos `while(op != 'x')` que permiten al usuario realizar múltiples operaciones sin tener que reiniciar el programa.

6. SEGURIDAD Y CONTROL DE ERRORES

6.1. Validación de Entrada

Enteros: La función `leer_entero` lee primero como string y valida con `isdigit` antes de convertir con `atoi`, evitando que el programa colapse si el usuario ingresa letras donde van números.

Integridad de Archivos: Antes de leer, siempre se verifica `if (!f)` para manejar el caso de archivos inexistentes.

6.2. Autenticación

El módulo `auth.c` actúa como portero.

Admin: Credencial dura

Médico: Detecta el prefijo o código en la base de datos de personal.

Paciente: Verifica existencia en `db_pacientes`.

7. REQUISITOS DE DESPLIEGUE

Para ejecutar el sistema en un entorno de producción:

Ejecutable: `MiProyecto.exe`.

Base de Datos: Los archivos `.txt` deben estar en el mismo directorio que el ejecutable.

Nota: Si los archivos .txt no existen, el sistema los creará automáticamente al registrar el primer dato.

7.1 acceso al código fuente

Para garantizar la transparencia, versionamiento y disponibilidad del proyecto, el código fuente completo, junto con los archivos de compilación y el ejecutable final, se han alojado en la plataforma GitHub.

Instrucciones de descarga:

1. Ingrese al siguiente enlace oficial del repositorio:

LINK: <https://github.com/Derlis-1/Citas-medicas.git>

2. Haga clic en el botón verde "**Code**".
3. Seleccione "**Download ZIP**".
4. Descomprima el archivo y ejecute MiProyecto.exe ubicado en la carpeta de ejecutable.